



# Binary Search Tree

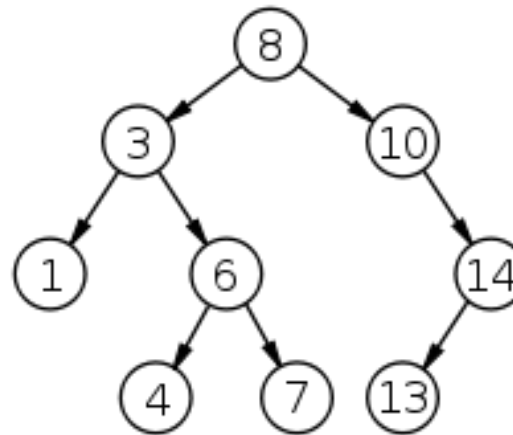


# Outlines

- Binary Search Tree
- BST Operations
- Insertion
- Deletion
- Complexity in BST
- Applications of BST

# Introduction :

- Binary Search Tree(BST), is a node-based binary tree data structure that store "items" (such as numbers, names etc.) in memory. BST sometimes called **ordered** or **sorted binary trees**.
- **Here is an example of BST :**





Binary Search Tree has the following properties:

- ☐ **The left subtree of a node contains only nodes with keys less than the node's key.**
- ☐ **The right subtree of a node contains only nodes with keys greater than the node's key.**
- ☐ **The left and right subtree each must also be a binary search tree.**
- ☐ **Each child must be leaf node or root of another binary**

The above properties of Binary Search Tree provide an ordering among keys so that the operations like search, minimum and maximum can be done fast.

# Binary Search Tree

In this Example:

2 is the root node .Its left child 1 which is smaller than root node & its right child 3 which is greater than right child .

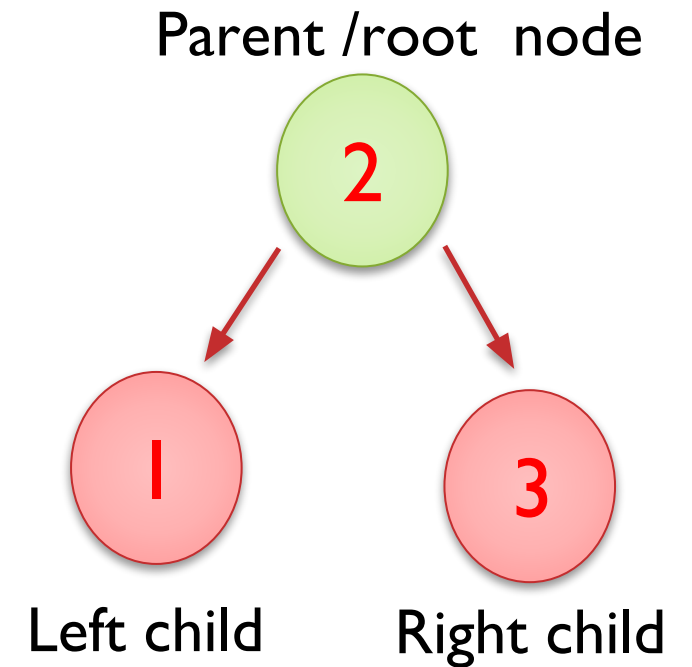


Fig : BST

Let's create a BST by inserting multiple keys into it .

# Insertion

8

3

1

9

6

14

□ Insert 8

8

3

1

9

6

14

# Insertion



# Insertion

□ Insert 8

8

3

1

9

6

14

□ Insert 3

8

# Insertion

3

1

9

6

14

□ Insert 3

8

# Insertion

3

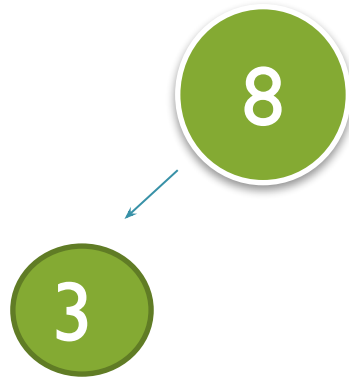
1

9

6

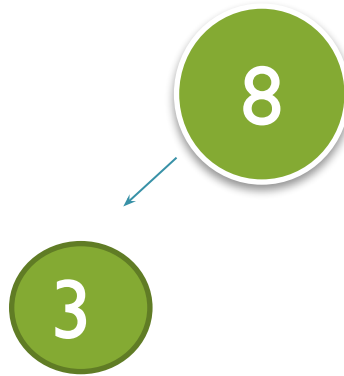
14

□ Insert 3



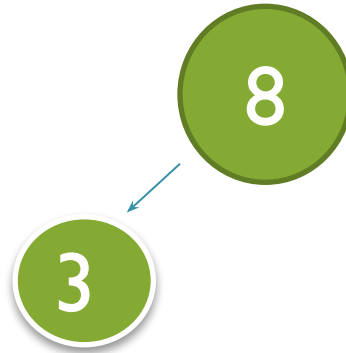
# Insertion

□ Insert 1



# Insertion

□ Insert 1



# Insertion

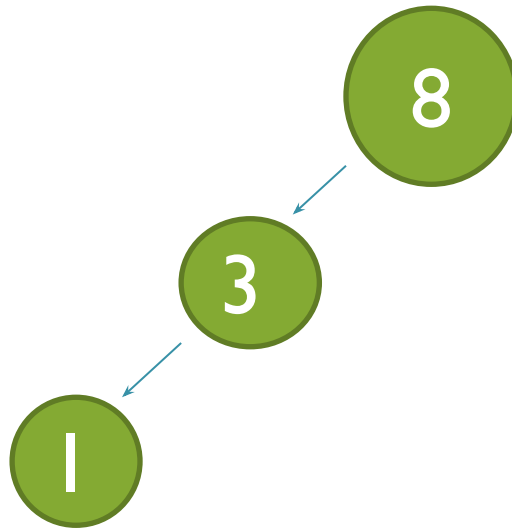
# Insertion

□ Insert 1

9

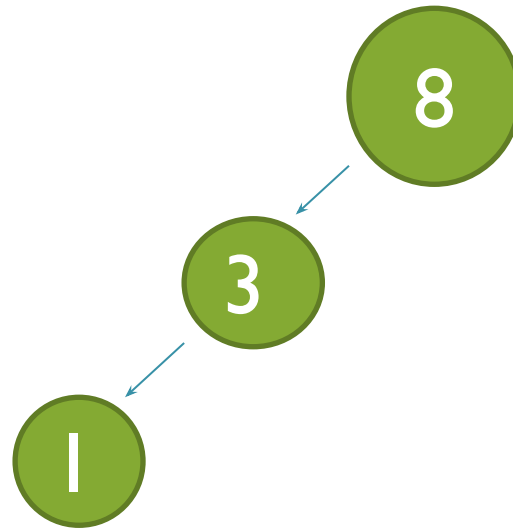
6

14



# Insertion

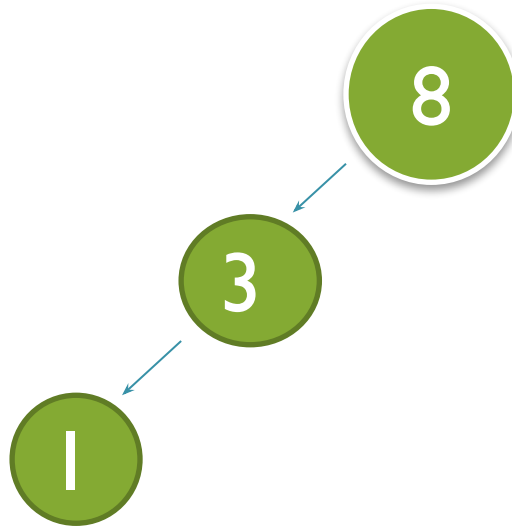
□ Insert 9





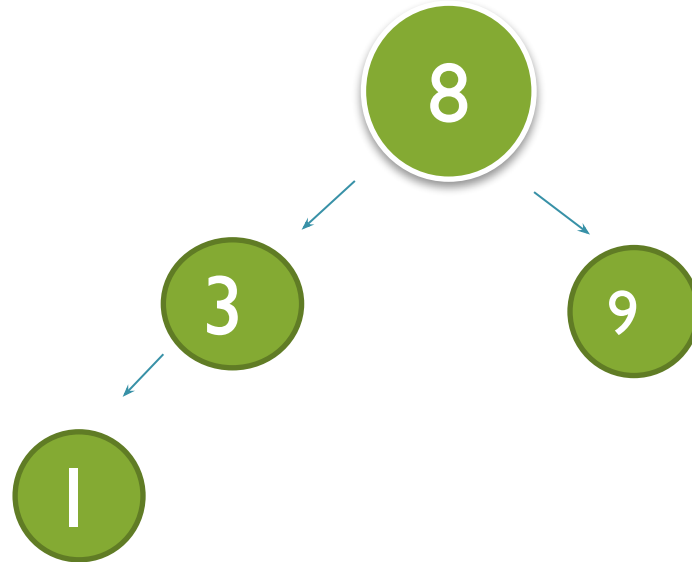
# Insertion

□ Insert 9



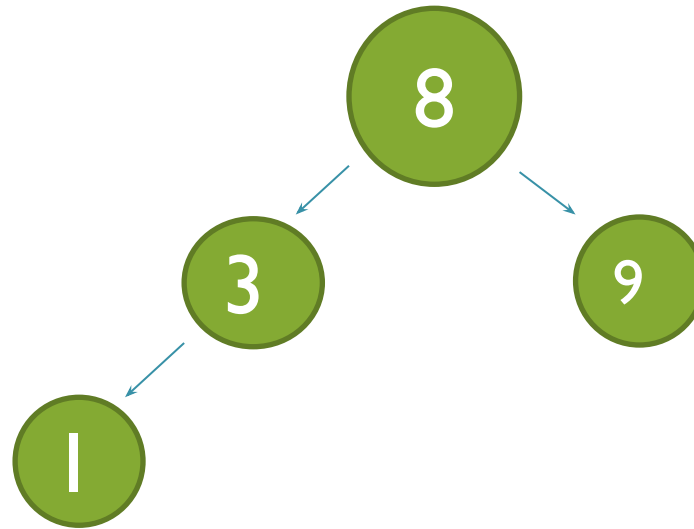
# Insertion

□ Insert 9



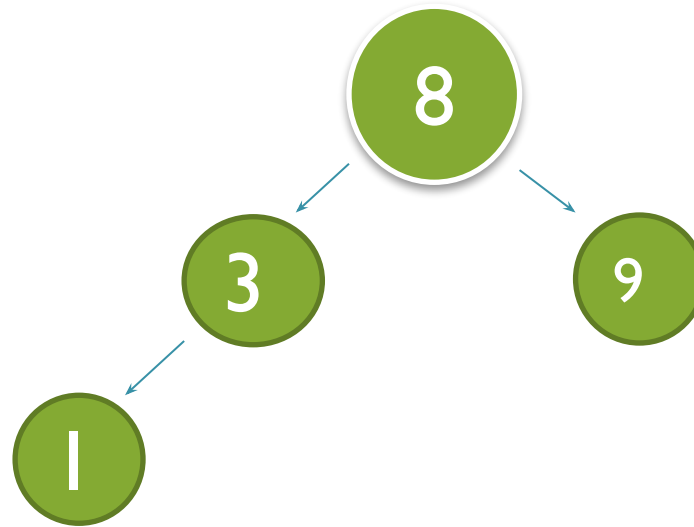
# Insertion

□ Insert 6



# Insertion

□ Insert 6

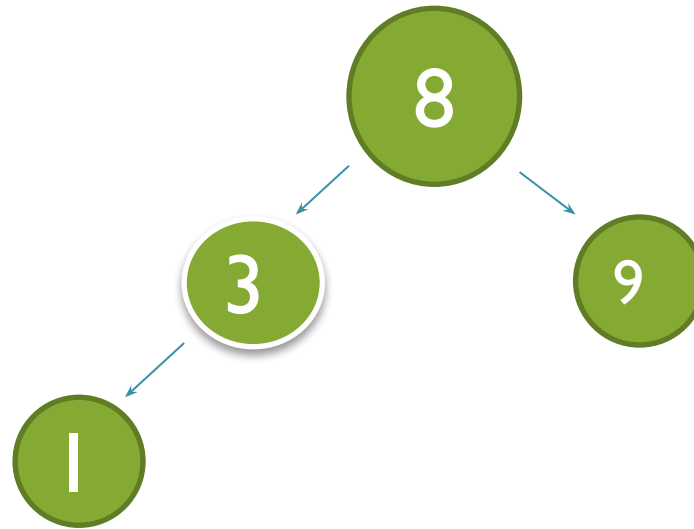


6

14

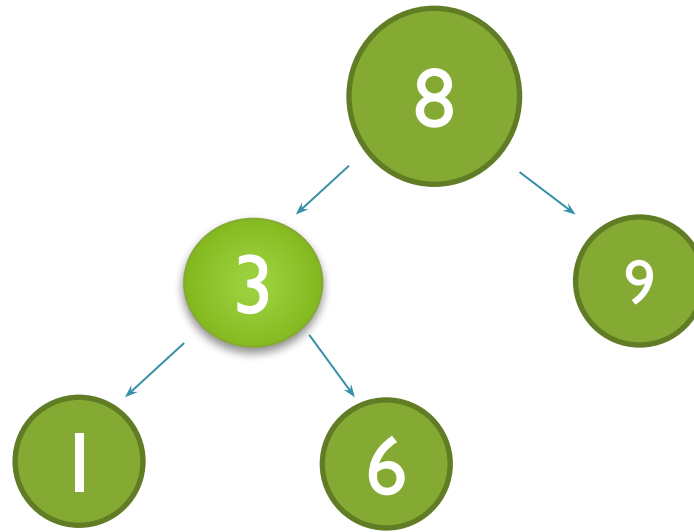
# Insertion

□ Insert 6

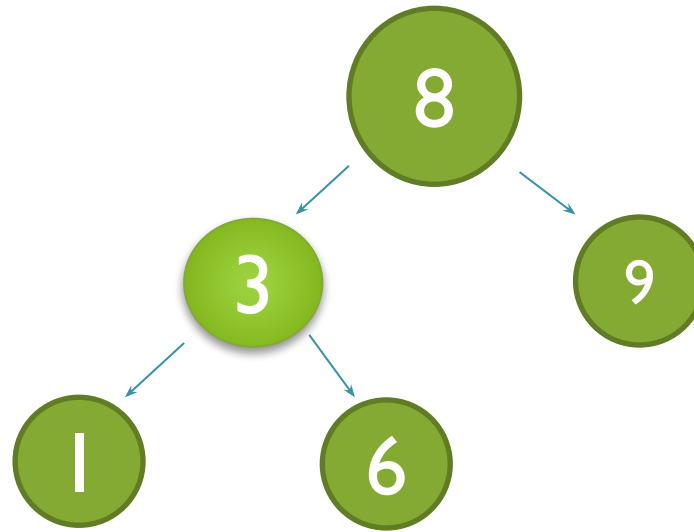


# Insertion

□ Insert 6



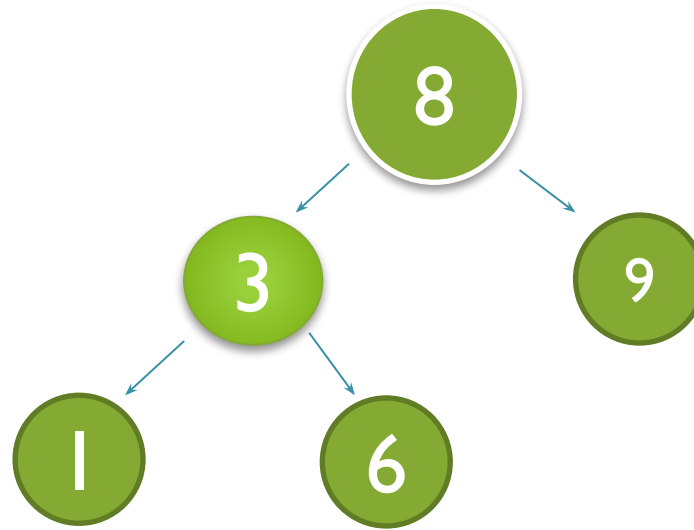
# Insertion



□ Insert 14



# Insertion

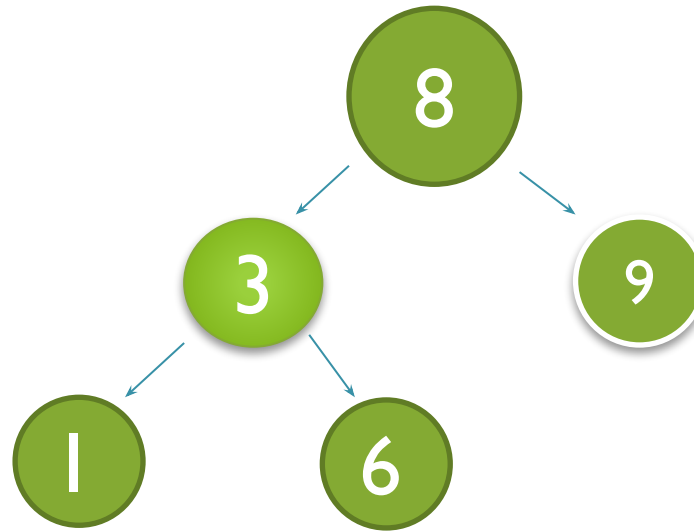


□ Insert 14





# Insertion

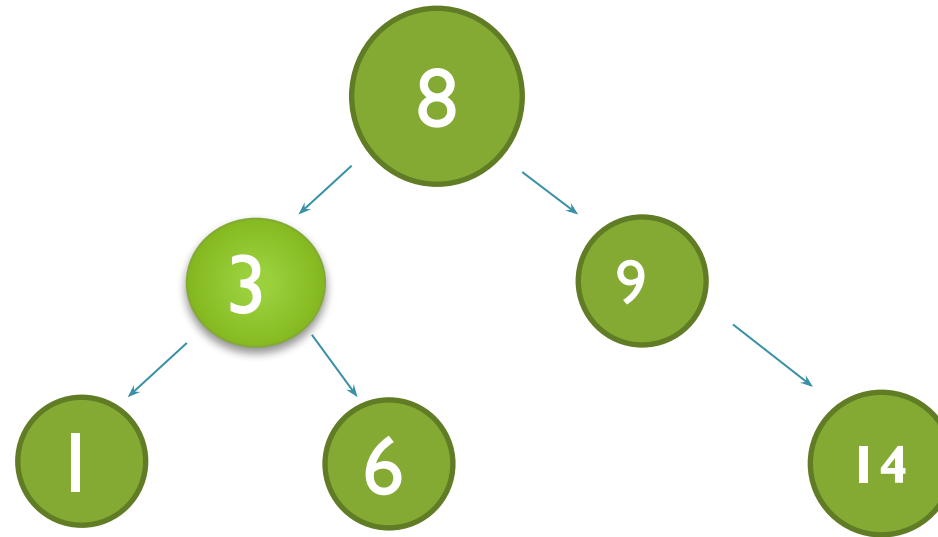


□ Insert 14



# Insertion

□ Insert 14





DELETION

## **What is deletion?**

Deletion means removing a node from a tree.

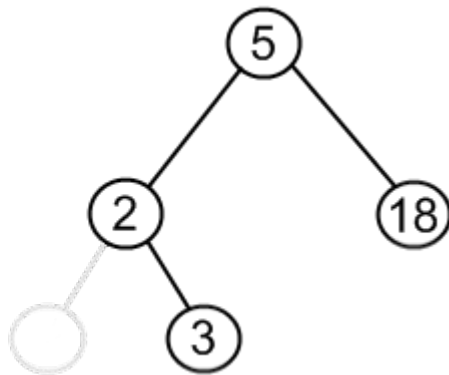
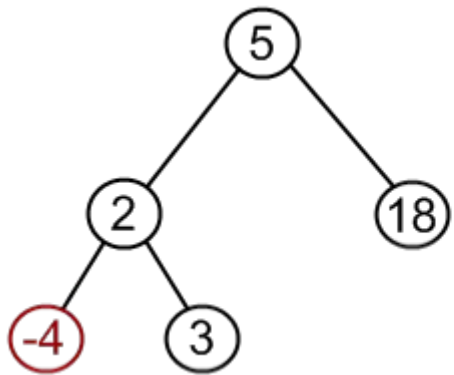
In BST there are three kinds of node...

- 1.node with no child.
- 2.node with one child.
- 3.node with two child.

So to delete a node from a BST we have to consider what kind of node we are

Deleting.

# No child node deletion

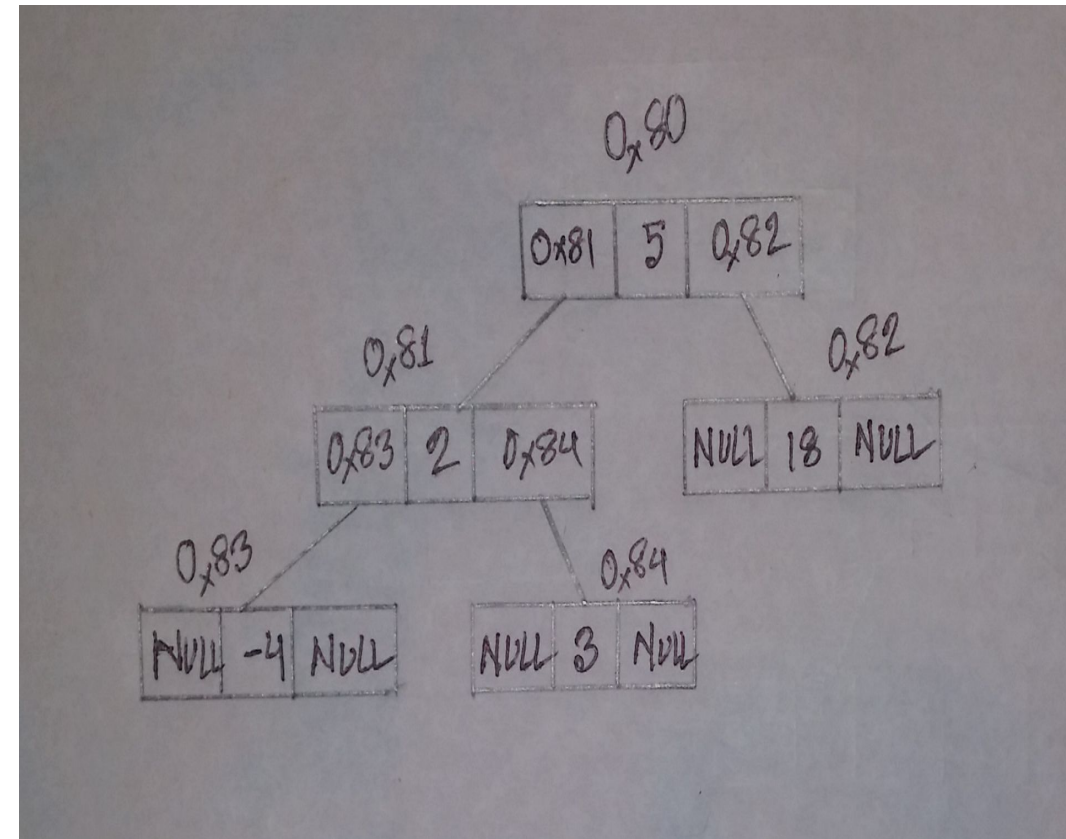


```

void find_node(int data)
{
    bst *str,*ptr;
    str=root; ptr=root;
    while(ptr!=NULL){
        if(ptr->information==data)
            break;
        else if(ptr->information >
data)
            { str=ptr;
              ptr=ptr->left;}
        else
            { str=ptr;
              ptr=ptr->right;}}}}

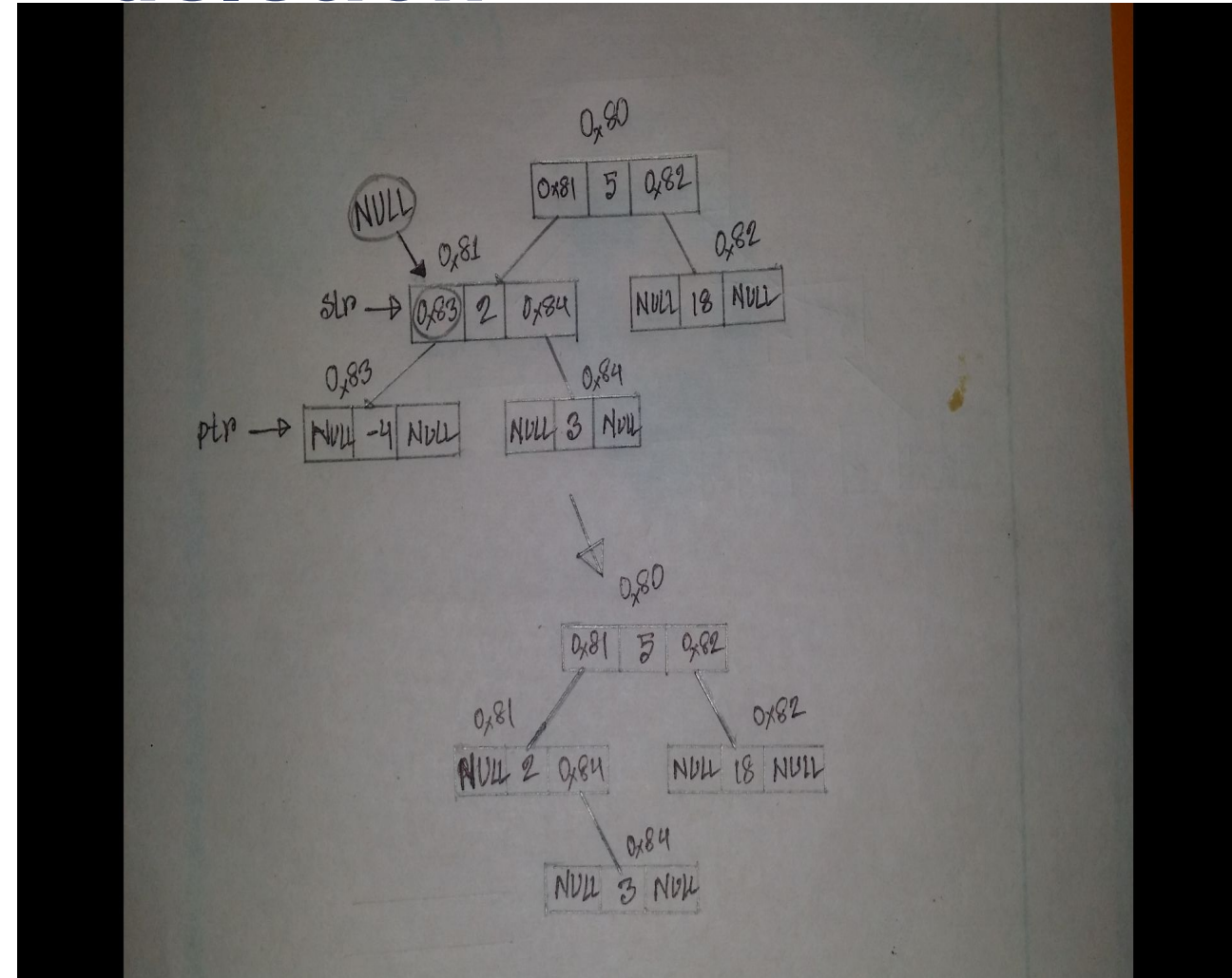
```

# How to find the node we want to delete?



# No child node deletion

- //code to delete a node having no child:
- //Here ptr holds the address of the node we want to delete:
- //str holds the address of ptr's parant:
- The condition for what a node is considered having no child:
- `If(ptr->right==NULL&&ptr->left==NULL)`
- {
- `If(str->left==ptr)`
- `str->left=NULL;`
- else
- `str->right=NULL;`
- }

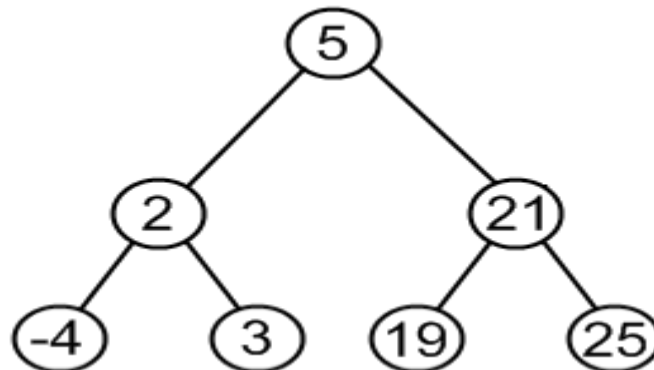
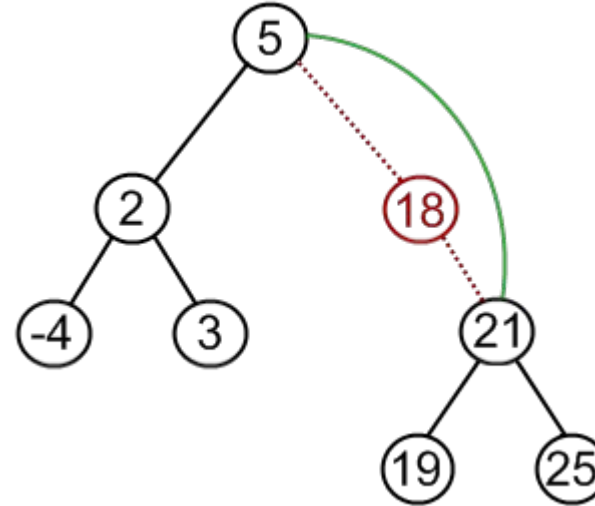
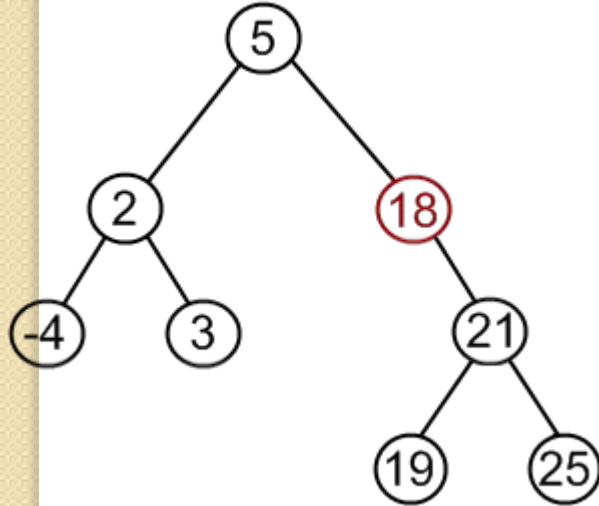


# One child node deletion

Node to be removed has one child.

It this case, node is cut from the tree and algorithm links single child (with it's subtree) directly to the parent of the removed node.

**Example.** Remove 18 from a BST.



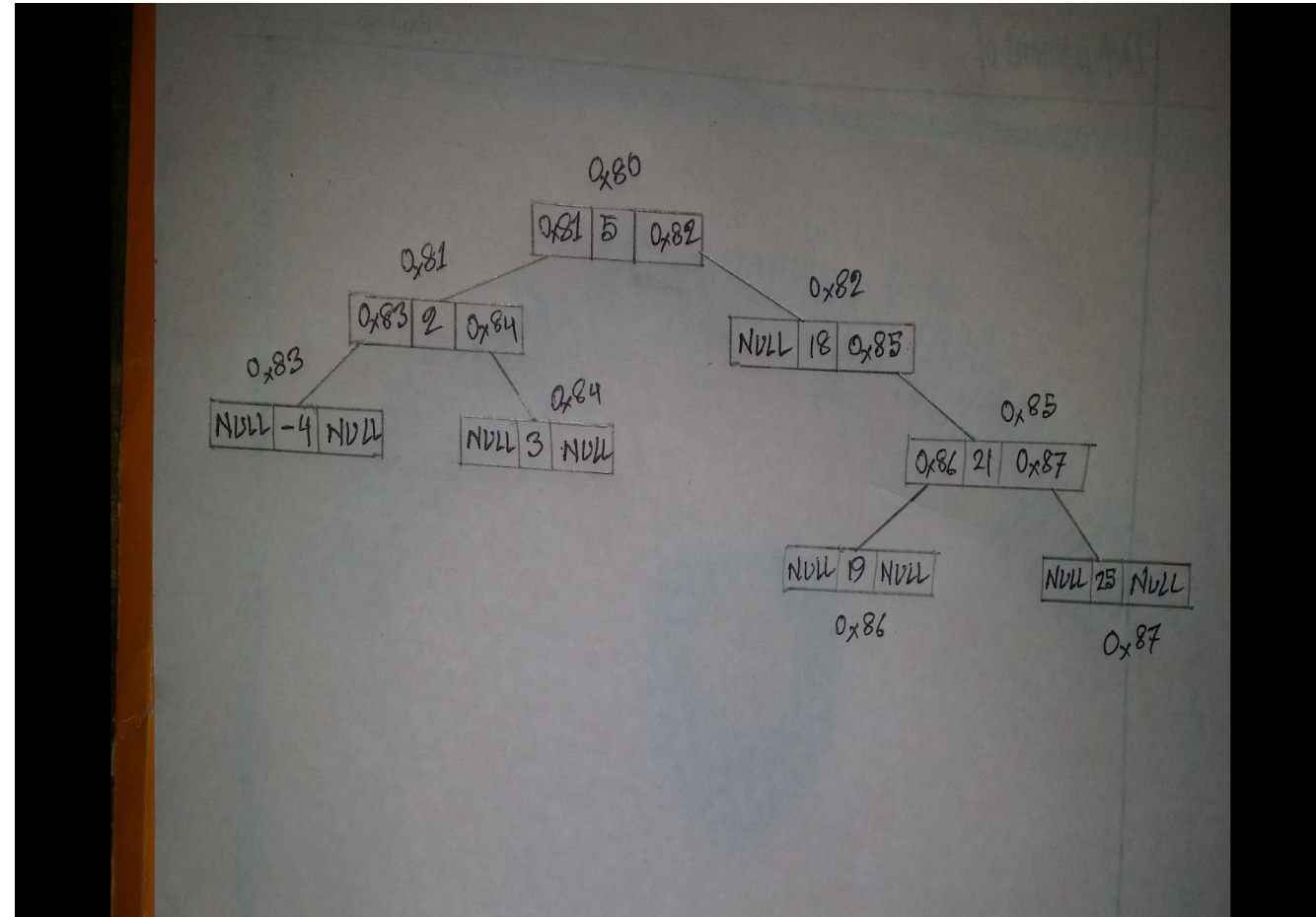


```

//code to find a node:
void find_node(int data)
{
    bst *str,*ptr;
    str=root; ptr=root;
    while(ptr!=NULL)
    {
        if(ptr->information==data)
            break;
        else if(ptr->information >
data)
            { str=ptr;
              ptr=ptr->left;}
        else
            {str=ptr;
              ptr=ptr->right;}}
    }

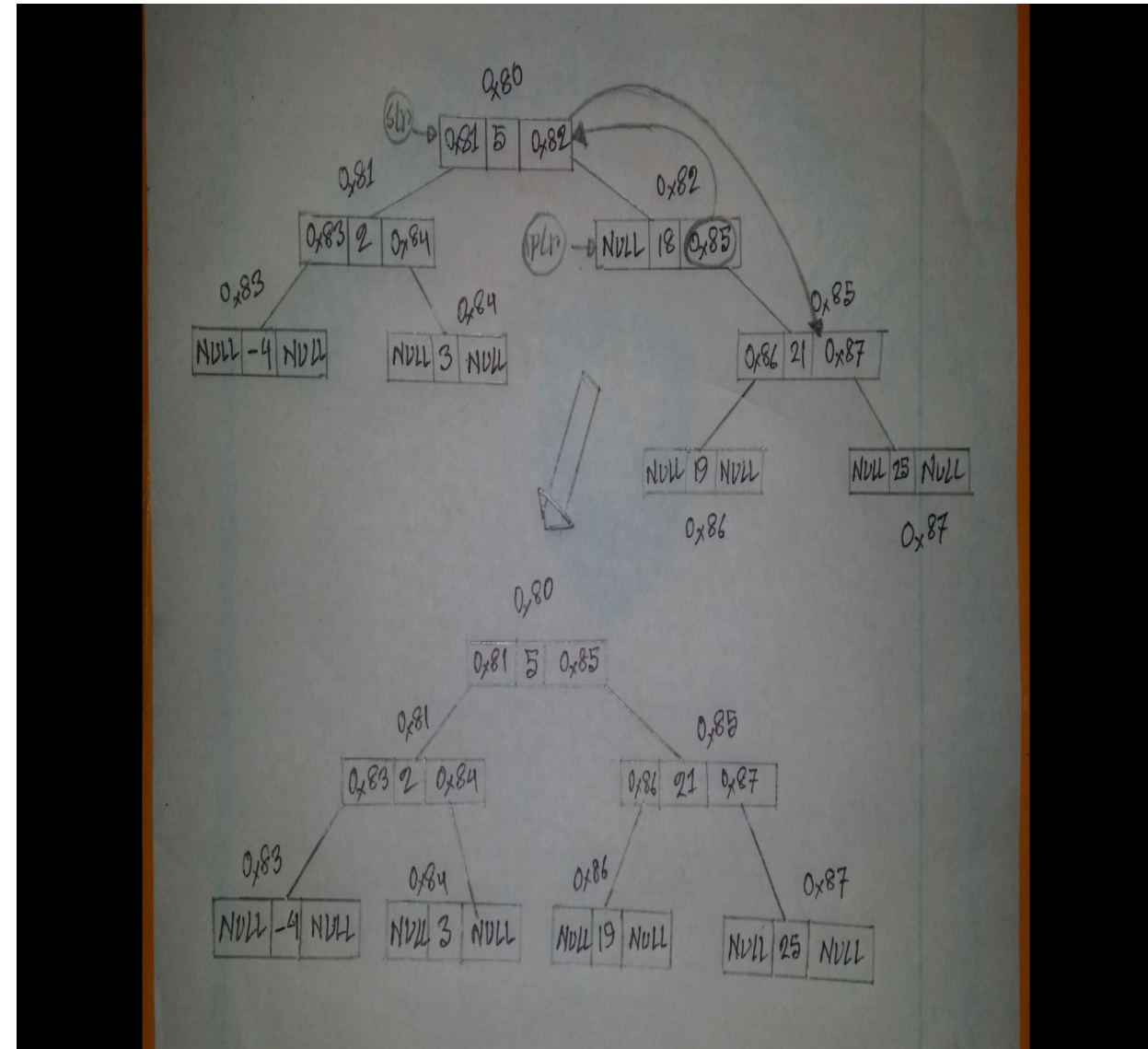
```

# How to find the node we want to delete?



# One child node deletion

- //Here ptr holds the address of the node we want to delete:
- //str holds the address of ptr's parent:
- The condition for what a node is considered having one child:
- $\text{If}((\text{ptr} \rightarrow \text{left} == \text{NULL} \ \& \ \text{ptr} \rightarrow \text{right} \neq \text{NULL}) \ || \ (\text{ptr} \rightarrow \text{left} \neq \text{NULL} \ \& \ \text{ptr} \rightarrow \text{right} == \text{NULL}))$
- { if(str->left==ptr)
- { if(ptr->left!=NULL)
- str->left=ptr->left;
- else
- str->left=ptr->right;}
- else if(str->right==ptr) {
- if(ptr->left!=NULL)
- str->right=ptr->left;
- else
- str->right=ptr->right; }
- }



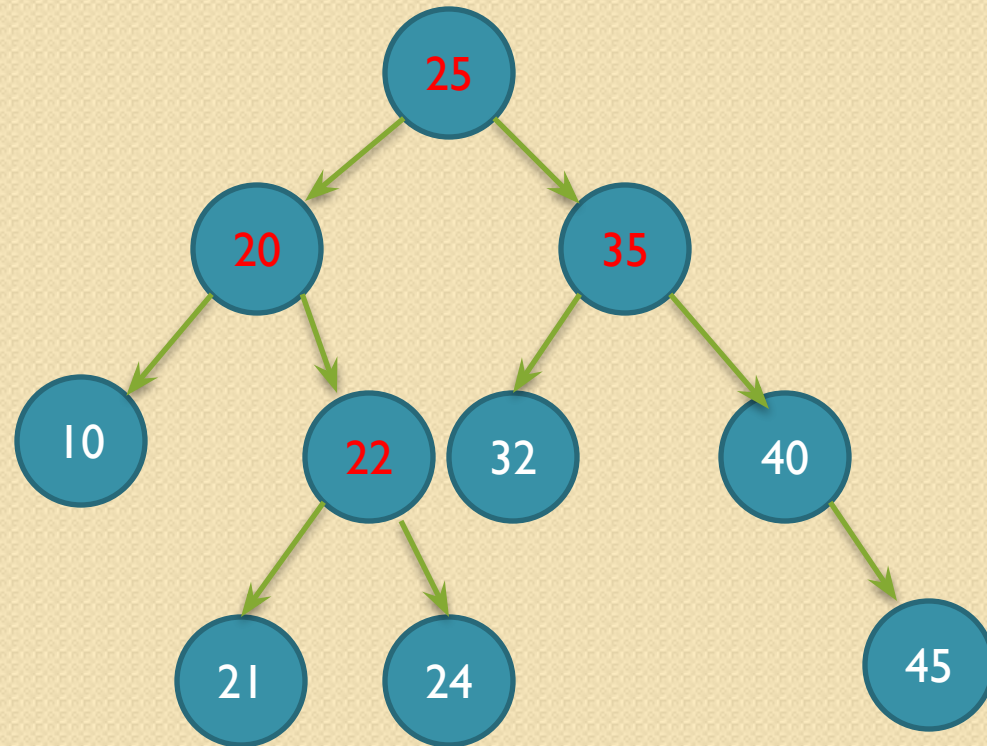
Now we are going to discuss about the node that has two child



## **NODE WITH TWO CHILD**

## NODE WITH TWO CHILD

In this BST the node that contain value 25,20,22,35 are the node with two child. Deleting any of them, we have to find the inorder successor of the node then have to replace it. And the inorder successor node have to be deleted from it's previous position.



# Node with two child

- How to delete it?

After tracking the element that has to be deleted from the BST, we have to check that if it has two child or not. If it has two child then we will go to the next instruction of algorithm.

When we delete the node, we have to store it's root node, it's two child node and it's inorder successor node.

# Node with two child

- First, we have to find it's inorder successor node.
- Then, connect it with the root node of the deleting node.
- After that we have to connect inorder successor node with the child of the deleting node.
- Finally, have to delete the inorder successor from it's previous position.

# Node with two child

For this BST it's inorder traversing output is :

10 , 20 , 21 , 22 , 24 , 25 , 32 , 35 ,  
40 , 45

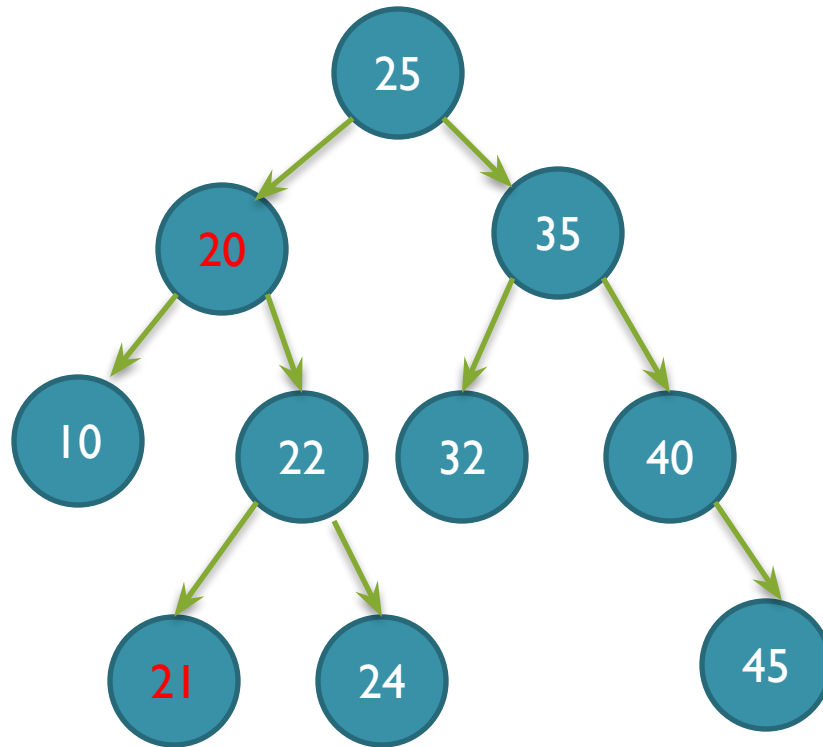
So, if we want to delete 20, it will replace by 21. so as, for 22 it will be 24, for 25 it will be 32, for 35 it will be 40.

```
//sample code to find inorder successor
//ptr node contain the deleting node address
a3 = ptr->right;
srt = ptr->right;
while (1)
{
    if ( srt->left == NULL )
        break; // srt node will contain the
    else      inorder successor
    {
        a3 = srt;//a3 node will contain the
        previous node of deleting node
        srt = srt->left;
    }
}
```

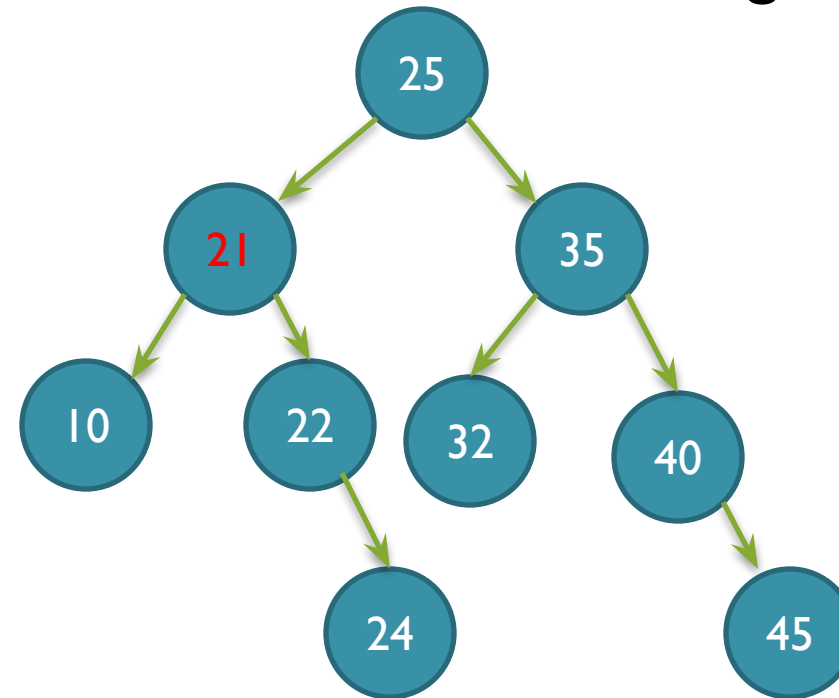


# Node with two child

- Before deleting 20 from BST



- After deleting 20 from BST





# Node with two child

//sample code to delete the two child  
containing node.

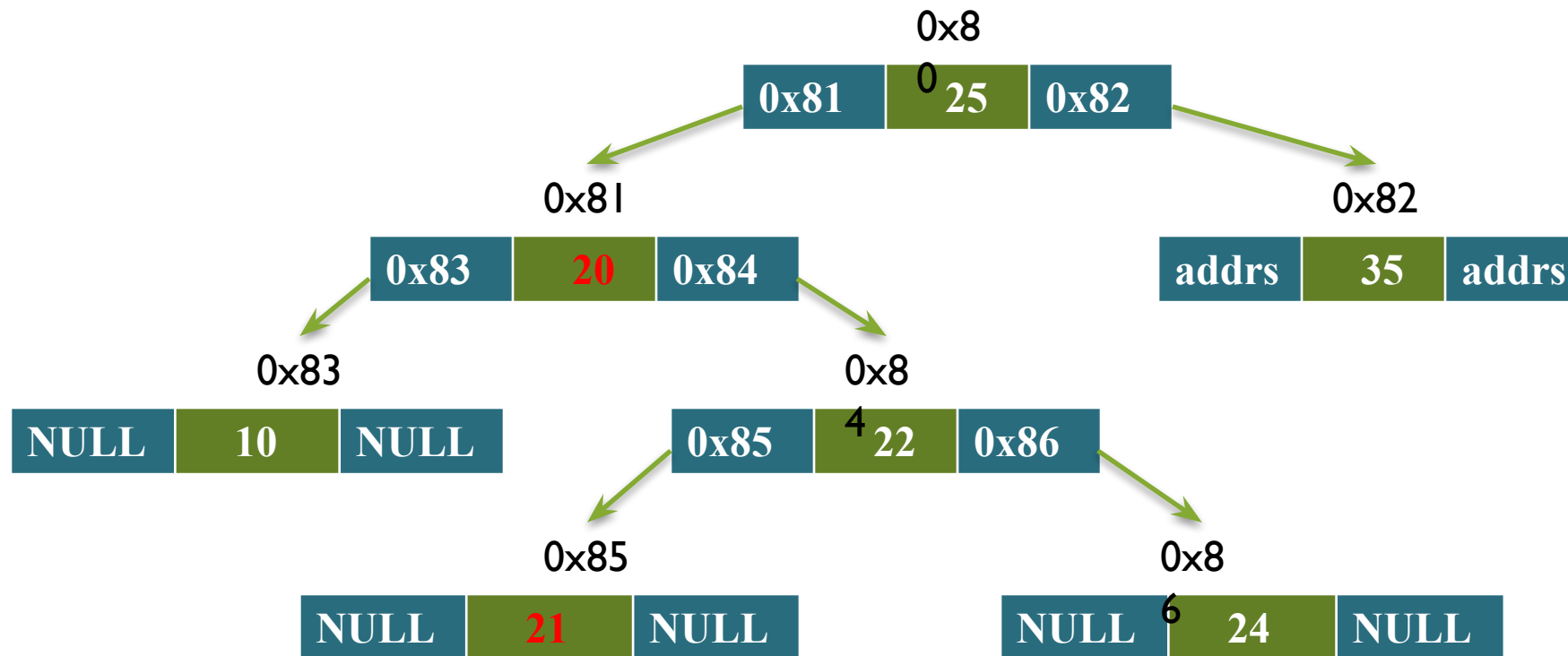
```
a1 = ptr->left ;  
//a1 node contain the left child of  
deleting node  
a2 = ptr->right ;  
//a2 node contain the right child of  
deleting node  
If ( tmp->left == ptr)  
    tmp->left = srt;  
else  
    tmp->right = srt;
```

//tmp node contain the root node of  
deleting node

```
if ( srt->info == a2->info )  
    {  
        srt->left = a1 ;  
    }  
else  
    {  
        srt->left = a1 ;  
        srt->right = a2 ;  
        a3->left = NULL;  
    }
```

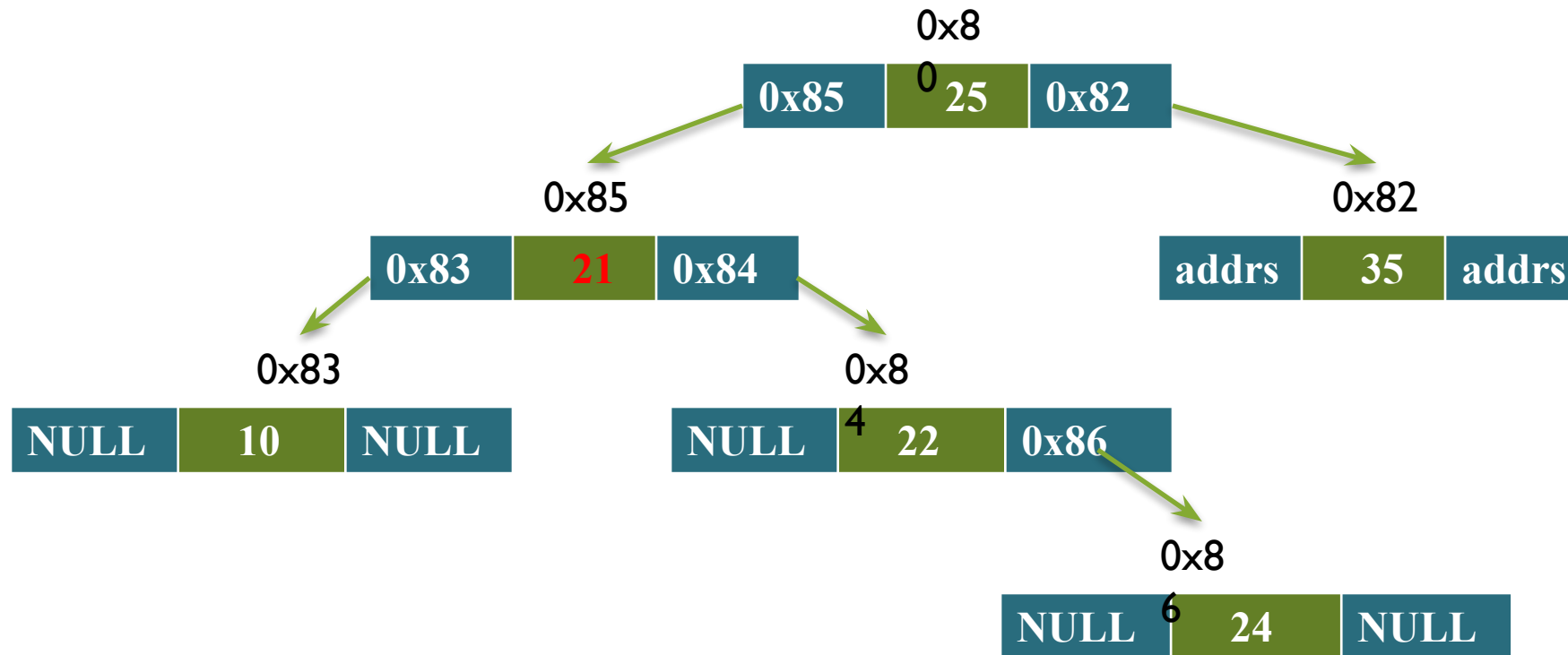
# Node with two child

Before deleting 20



# Node with two child

After deleting 20





# ***Complexity***



What happened to our little programmer??

reset

| Who     | Problem                                            | Lang      | Verdict                        |
|---------|----------------------------------------------------|-----------|--------------------------------|
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 15 |
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 15 |
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 15 |
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 15 |
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 25 |
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 25 |
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 25 |
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 25 |
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 25 |
| Mr_Troy | <a href="#">805F - Expected diameter of a tree</a> | GNU C++14 | Time limit exceeded on test 25 |

☐ Default order
 ☐ Submission time
 ☐ Judging Time
 ☒ Solution Si

# complexity

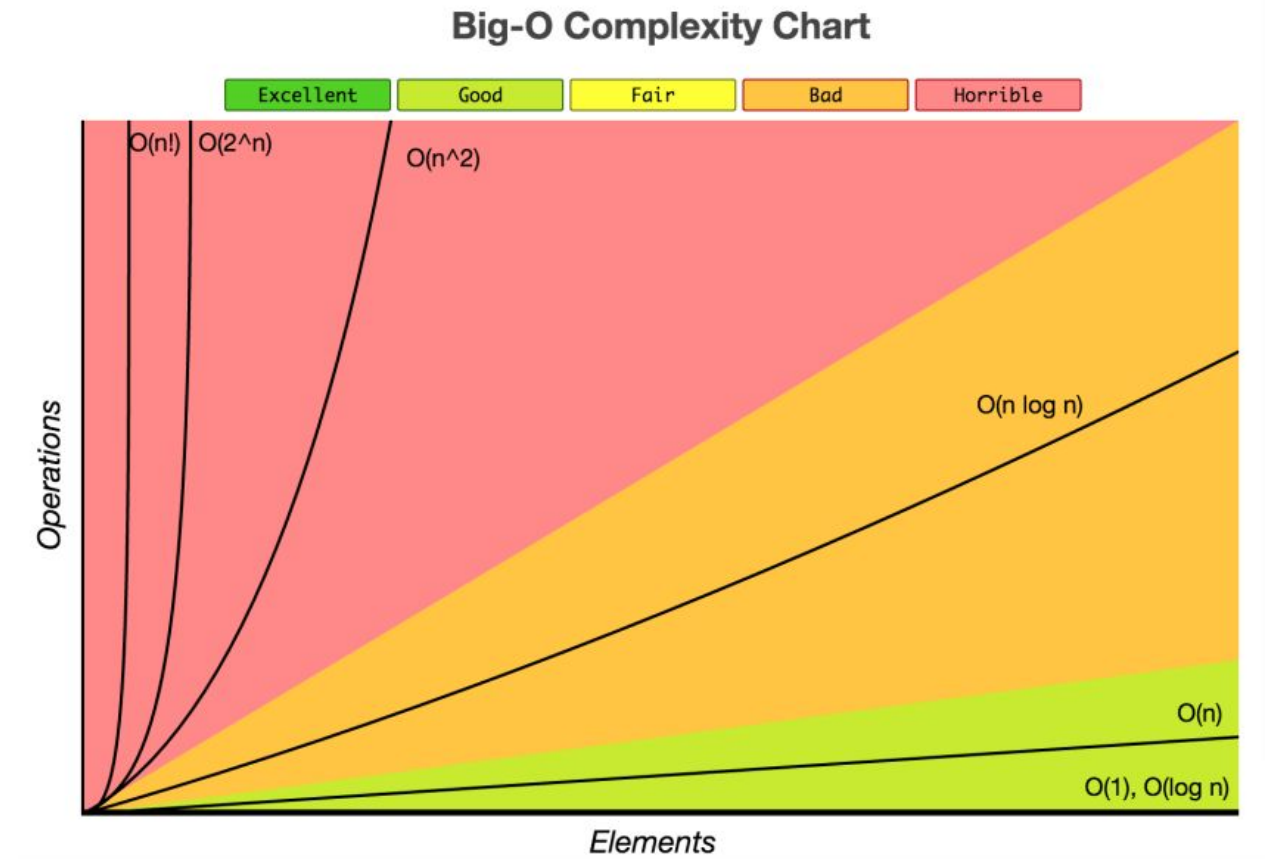
The term complexity refers to how fast or slow particular algorithm performs and how much space it is working on.

Types of complexity:

- Time complexity
- Space complexity

# Complexity

In computer science, big O notation is used to classify algorithms according to how their running time or space requirements grow as the input size grows



# Space complexity

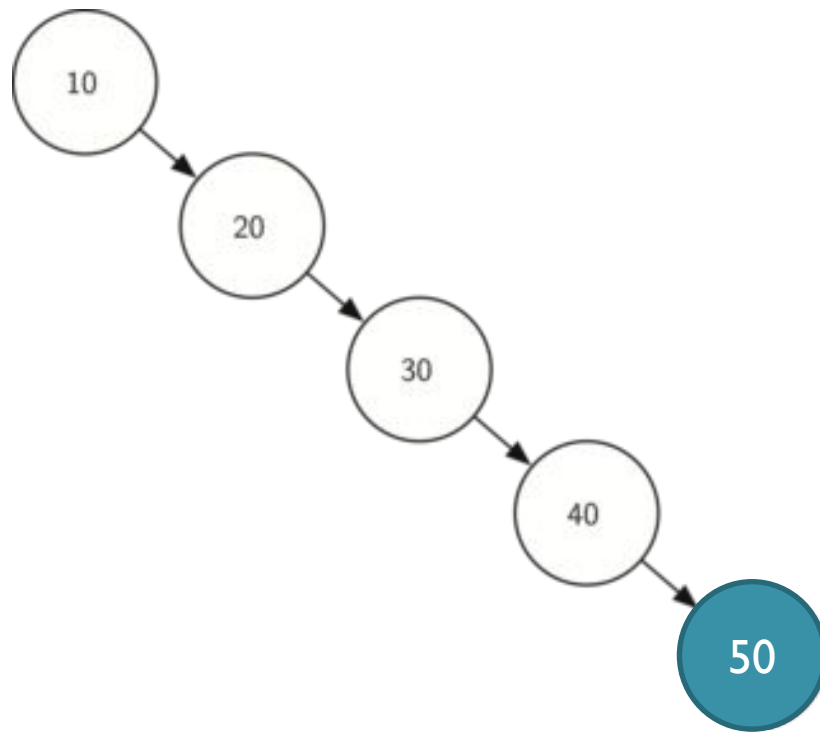
- ° **SPACE COMPLEXITY IS A MEASURE OF THE AMOUNT OF WORKING STORAGE AN ALGORITHM NEEDS**

So Tree having  $N$  nodes , has a complexity of  $O(n)$



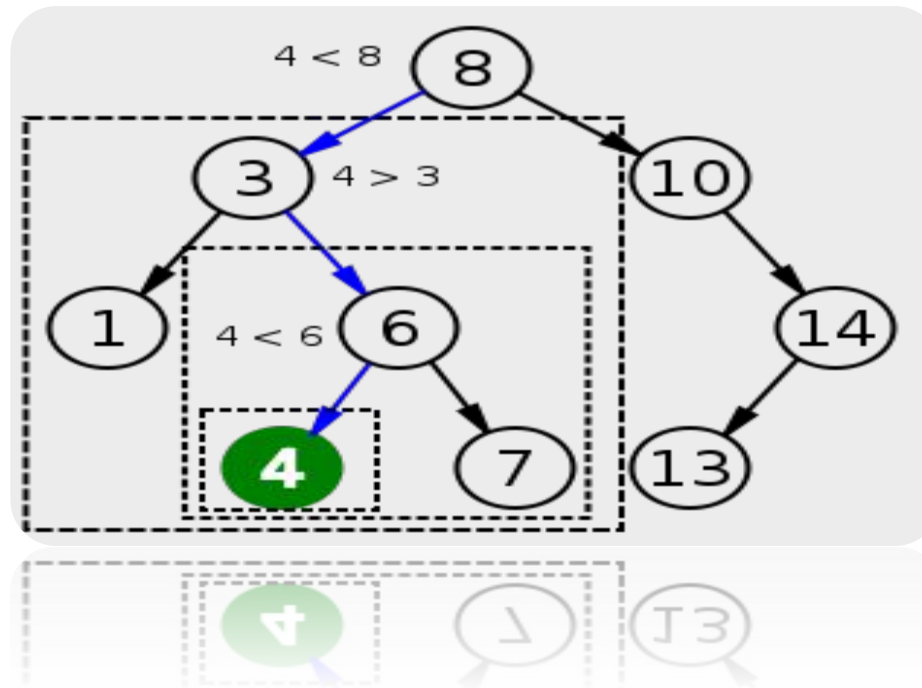
# Search

Worst case:



# Search

Avarage case:



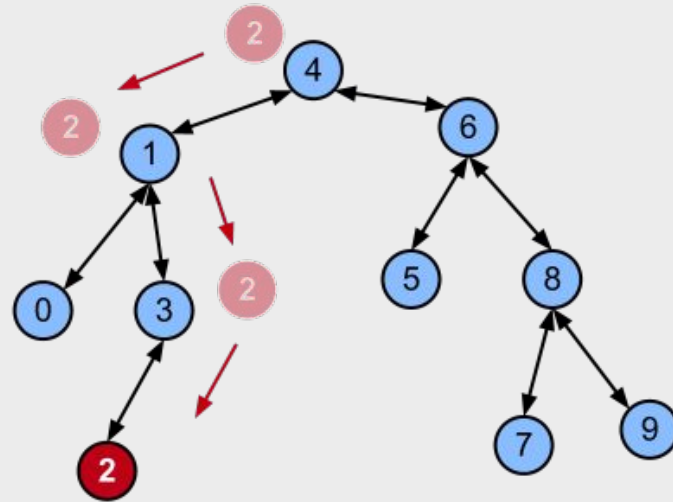
# Search

The time complexity for a single search in a balanced binary search tree is  $O(\log(n))$ . Maybe the question requires you to do  $n$  searches in the binary tree, hence the total complexity is  $O(n\log(n))$ .

The worst case complexity for a single search in an unbalanced binary search tree is  $O(n)$ . And similarly, if you are doing  $n$  searches in the unbalanced tree, the total complexity will turn out to be  $O(n^2)$ .

# Insertion

## Insert in BST



# Insertion

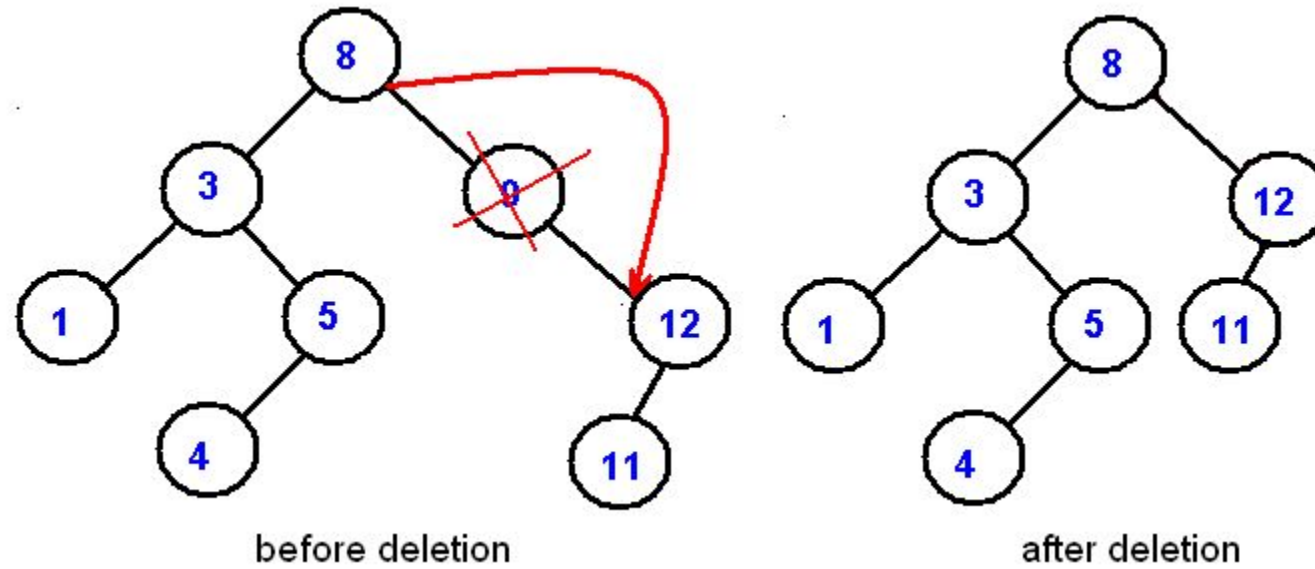
So , we can see for insertion search operation is going through the program. So the time complexity will be same as search.

In average case which is  $O(\log n)$ ,

In best case it is  $O(1)$ ;

In worst case it is  $O(n)$ .

# Deletion



# Deletion

Here also we can see for deletion search operation is going through the program. So the time complexity will be same as search.

In average case which is  $O(\log n)$ ,

In best case it is  $O(1)$ ;

In worst case it is  $O(n)$ .

# Overview

|                  | operation | Best case | Avarage case | Worst case |
|------------------|-----------|-----------|--------------|------------|
| Space complexity |           | $O(1)$    | $O(n)$       | $O(n)$     |
| Time complexity  | INSERTION | $O(1)$    | $O(\log n)$  | $O(n)$     |
|                  | SEARCHING | $O(1)$    | $O(\log n)$  | $O(n)$     |
|                  | DELETION  | $O(1)$    | $O(\log n)$  | $O(n)$     |





OH. Now he is happy...

# **Applications of BST**

# Applications Of BST

- Used in many search applications where data is constantly entering/leaving , such as the map and set objects in many languages' libraries.
- Storing a set of names , and being able to lookup based on a prefix of the name . (Used in internet routers.)
- Storing a path in a graph , and being able to reverse any subsection of the path in  $O(\log n)$  time . (Useful in travelling salesman problems.)

# Application of BST

- Finding square root of given number.
  - Allows to do range searches efficiently .(Like in Dictionary)
  - To implement multilevel indexing in DATABASE .

# Applications Of BST

- To implement Huffman Coding Algorithm
- To implement simple sorting algorithms such as Heap Sort
- Serves as priority queues structures that allow insertion of arbitrary key as well as lookup and deletion of the minimum (or maximum) key.

# Web References

- <https://www.slideshare.net/shimulsakhawat/binary-search-treebst>
- <https://github.com/tim-hr/stuff/wiki/Time-complexity:-Binary-search-tree>
- <http://bigocheatsheet.com/>
- [https://en.wikipedia.org/wiki/Binary\\_search\\_tree#cite\\_note-2](https://en.wikipedia.org/wiki/Binary_search_tree#cite_note-2)

Thank You